

SEMANA 3 — DÍA 3

Manejo profesional de excepciones en Python para Script 1 y Script 2

 **Objetivo del día:** Comprender y aplicar correctamente el manejo de excepciones en Python (try, except, else, finally) para hacer que el Script 1 y Script 2 sean robustos, seguros y resistentes a fallos reales.

TEORÍA

1. ¿Qué es una excepción?

Una **excepción** es un error que ocurre durante la ejecución del programa y que **interrumpe el flujo normal** si no se maneja.

Ejemplos comunes:

- La API no responde
- El archivo no existe
- El JSON está mal formateado
- Un campo no existe
- Se intenta convertir texto a número
- Se divide entre cero

Python detiene el programa a menos que tú **atrapes** la excepción.

2. Estructura completa de manejo de excepciones

Python tiene cuatro bloques:

```
try:
    # Código que puede fallar
except:
    # Qué hacer si falla
else:
    # Qué hacer si NO falló
finally:
    # Qué hacer siempre, falle o no falle
```

✓ 2.1 try — donde ocurre el riesgo

Aquí va el código que podría fallar:

```
try:
    response = requests.get(url)
```

✓ 2.2 except — qué hacer si ocurre un error

Puedes capturar:

- Errores específicos
- Errores generales
- Múltiples errores

Ejemplo:

```
except requests.exceptions.ConnectionError:
    print("No hay conexión a internet.")
```

O general:

```
except Exception as e:
    print("Ocurrió un error:", e)
```

✓ 2.3 else — se ejecuta solo si NO hubo error

Ideal para:

- Procesar datos
- Validar
- Limpiar

Ejemplo:

```
else:  
    data = response.json()
```

✓ 2.4 finally — se ejecuta SIEMPRE

Útil para:

- Cerrar archivos
- Liberar recursos
- Imprimir mensajes finales

Ejemplo:

```
finally:  
    print("Proceso terminado.")
```

3. Excepciones comunes en Script 1

✓ 3.1 Conexión a la API

```
requests.exceptions.ConnectionError  
requests.exceptions.Timeout  
requests.exceptions.HTTPError
```

✓ 3.2 JSON mal formateado

```
json.JSONDecodeError
```

✓ 3.3 Campos inexistentes

```
KeyError
```

✓ 3.4 Conversión de tipos

```
ValueError  
TypeError
```

✓ 3.5 Archivos faltantes

```
FileNotFoundError
```

4. Excepciones comunes en Script 2

✓ 4.1 Estadística

```
statistics.StatisticsError    # listas vacías, sin moda, etc.  
ZeroDivisionError            # división entre cero
```

✓ 4.2 Visualizaciones

```
ValueError    # listas de diferente longitud  
TypeError     # valores no numéricos
```

✓ 4.3 Guardado de archivos

```
FileNotFoundError  
PermissionError
```

Ejemplos

Manejo de errores en api_client.py

```
import requests  
import json  
  
def get_data(url):  
    try:  
        response = requests.get(url, timeout=10)  
        response.raise_for_status() # lanza error si status_code no es 200  
    except requests.exceptions.Timeout:  
        print("La API tardó demasiado en responder.")  
        return None  
    except requests.exceptions.ConnectionError:  
        print("Error de conexión. Verifica tu internet.")  
        return None  
    except requests.exceptions.HTTPError as e:  
        print("Error HTTP:", e)  
        return None  
    else:  
        try:  
            return response.json()  
        except json.JSONDecodeError:  
            print("La API devolvió un JSON inválido.")  
            return None  
    finally:  
        print("Intento de conexión finalizado.")
```

Manejo de errores al leer JSON limpio

```
import json  
  
def cargar_datos_limpios():  
    try:  
        with open("data/clean/data_clean.json", "r") as f:  
            return json.load(f)  
    except FileNotFoundError:  
        print("El archivo limpio no existe. Ejecuta primero el Script 1.")  
    except json.JSONDecodeError:  
        print("El archivo limpio está corrupto.")  
    return None
```

Manejo de errores en análisis

```
import statistics as stats  
  
def calcular_media(lista):  
    try:  
        return stats.mean(lista)  
    except stats.StatisticsError:  
        print("No se puede calcular la media de una lista vacía.")  
        return None
```

Manejo de errores en visualizaciones

```
import matplotlib.pyplot as plt

def graficar_linea(x, y):
    try:
        plt.plot(x, y, marker="o")
        plt.title("Gráfica de línea")
        plt.savefig("figures/linea.png")
        plt.show()
    except ValueError:
        print("Las listas x e y deben tener la misma longitud.")
    except Exception as e:
        print("Error al generar la gráfica:", e)
```

ACTIVIDAD EN CLASE

Cada equipo debe:

✓ 1. Revisar su Script 1 y Script 2

Identificar dónde podrían ocurrir errores reales.

✓ 2. Implementar manejo de excepciones en:

- api_client.py
- validators.py (regex mal aplicadas)
- cleaner.py (conversiones de tipo)
- analysis.py (listas vacías, estadísticas)
- visualizations.py (listas de diferente tamaño, valores no numéricos)
- script1.py y script2.py (flujo completo)

✓ 3. Crear un archivo errors.md con:

- Lista de errores que encontraron
- Cómo los manejaron
- Ejemplos de código

✓ 4. Subir todo a GitHub.

TAREA

- Terminar errors.md.
- Revisar que todos los módulos tengan manejo de excepciones.
- Probar Script 1 y Script 2 con datos incorrectos para validar robustez.